Pallas: Generic Asset Administration Shell-Based Representation of Planning Knowledge for Automatic PDDL Generation

Keywords: AI Planning, PDDL, Asset Administration Shell, Semantic Planning Knowledge, Industry 4.0, Knowledge Engineering, Planning Domain Generation

Abstract: Artificial Intelligence (AI) planning provides powerful methods for automated decision-making, but practical adoption remains limited by challenges such as the effort required to create and maintain formal planning models. In practice, planning knowledge is often encoded directly in the Planning Domain Definition Language (PDDL), which couples domain knowledge to planner-oriented syntax. Plato addressed this with an ontology-based model for semantic planning knowledge. This paper presents Pallas, an Asset Administration Shell (AAS)-based counterpart for Industry I4.0 environments. Pallas represents types, objects, actions, and related planning concepts through standard AAS submodels and supports distributed modeling across multiple AASs. The Knowledge Transformation Framework operationalizes Pallas models by merging distributed information, applying planner-specific transformations, and producing PDDL files. An initial proof-of-concept evaluation with four example domains and an exploratory user study indicates Pallas feasibility, while larger case studies remain future work.

1 INTRODUCTION

Digital transformation increasingly requires systems that generate, revise, and coordinate decisions under resource, temporal, and safety constraints. Such requirements occur in various domains, including autonomous space operations and planetary exploration (Muscettola et al., 1998; Bresina et al., 2005), urban traffic management (McCluskey et al., 2017), industrial manufacturing (Malburg et al., 2023; Wally et al., 2019), robotics (Alterovitz et al., 2016), and even video games (Bartheye and Jacopin, 2009).

Artificial Intelligence (AI) planning is a prominent approach for this form of automated decision-making (Green, 1969; Haslum, 2006). A AI planning application requires a planning model, a representation language, and a planner (Marrella, 2019). The model captures semantic information about the domain; the language encodes it machine-readable form; and the planner searches for a solution. The Planning Domain Definition Language (PDDL) is the de facto standard for encoding planning domains and problems (Haslum et al., 2019; Helmert, 2009; Ammar and Bhiri, 2018).

Despite its potential, practical planning deployment remains difficult. One central obstacle is its knowledge-intensive nature (Alnazer and Georgievski, 2023). Required domain knowledge must be acquired, formalized, validated, and maintained in an explicit model (Malburg et al., 2023). Manual modeling is time-consuming, error-prone, and reflects the classical knowledge acquisition bottleneck of knowledge-based AI (Feigenbaum, 1977; Studer et al., 1998), especially when resources, capabilities, or process constraints change over time.

Semantic source models reduce this effort by separating planning knowledge from planner-specific artifacts such as PDDL. Malburg et al. (Malburg et al., 2023) showed this principle by transforming semantic web service descriptions and ontological knowledge into PDDL for manufacturing planning and scheduling. Plato generalizes the idea with an ontology schema for semantic planning knowledge and automatic PDDL generation (Plato, 2026) . It is well suited for environments where semantic technologies, such as the Resource Description Framework (RDF) and the Web Ontology Language (OWL), and ontology engineering tools are already available and commonly used.

At the same time, Asset Administration Shells (AASs) (Industrial Digital Twin Association and Plattform Industrie 4.0, 2024) are increasingly important, especially in German and European Industry 4.0 (I4.0) contexts, where AI, data analytics, the Internet of Things (IoT), and distributed systems are embedded in industrial workflows (Gilchrist, 2016). An AAS standardizes the digital representation of an asset and organizes information in submodels. Since AASs already describe machine characteristics, capabilities, resources of production facilities (Bernhard

et al., 2024; Köcher et al., 2023), they are a natural place for planning knowledge when the relevant assets already have digital representations.

Pallas addresses this setting with a generic AAS-based model for semantic planning knowledge and automatic PDDL generation. It complements Plato rather than replacing it: Plato fits ontology-centered infrastructures, whereas Pallas targets AAS-centered environments. Storing planning knowledge near the assets that provide the corresponding actions or services reduces integration effort and supports distributed maintenance. This is relevant because industrial assets often already have an individual AASs; a monolithic planning model would separate planning knowledge from the digital representations in which related asset knowledge is already maintained. Although motivated by I4.0, Pallas remains a generic model for planning domains beyond industrial use because AASs are semantic data structures rather than only manufacturing artifacts.

The paper contributes Pallas, describes its representation of planning constructs with AAS submodels, and shows how the Knowledge Transformation Framework (KTF) transforms these models into planner-specific PDDL. It also reports an initial evaluation through proof-of-concept transformations and an exploratory user study comparing Pallas with Plato. Section 2 introduces foundations and related work, Section 3 states requirements and methodology, Sections 4 and 5 present the model and tool support, Section 6 evaluates them, and Sections 7 and 8 discuss limitations and future work.

2 FOUNDATIONS AND RELATED WORK

This section introduces the planning and AAS foundations required for Pallas and positions the approach in relation to existing work.

2.1 AI Planning and PDDL

AI planning determines how a system can reach a goal from an initial state (Green, 1969; Haslum, 2006). In classical planning, states describe relevant world properties and actions describe possible changes through applicability conditions and effects. A planning problem consists of an initial state, a goal condition, and the available actions; a solution is usually a finite sequence of parameterized actions that reaches a goal state.

Blocks World is a standard example (Russell and Norvig, 2020). Blocks can be stacked, picked up, and placed on other blocks or on a table. An action such as picking up a block is applicable only if the block is clear; executing it changes the state, for example by making the block held. Although simple, the domain illustrates the concepts that also occur in more complex settings.

PDDL is the de facto standard language for describing planning domains and problems (Haslum et al., 2019; Helmert, 2009; Ammar and Bhiri, 2018). A domain file defines reusable information such as requirements, types, predicates, functions, constants, and actions. A problem file provides objects, the initial state, the goal, and possibly a optimization metric. PDDL also supports temporal and numeric extensions, including durative actions, time-scoped conditions or effects, and arithmetic expressions for costs, durations, and capacities (Fox and Long, 2003).

Planner support for PDDL fragments varies. A domain valid for one planner may not be accepted by another, especially when it uses temporal, numeric, or complex logical constructs. Maintaining knowledge directly in PDDL therefore couples domain modeling to planner syntax and capabilities. Pallas follows the motivation of semantic source models: maintain planning knowledge in a structured representation and compile it into planner-specific PDDL artifacts.

2.2 Asset Administration Shells

Asset Administration Shells are standardized digital representations of assets in I4.0 environments (Industrial Digital Twin Association and Plattform Industrie 4.0, 2024). They can represent physical and non-physical assets and organize information about them. Pallas uses standard AAS submodel elements. *Properties* store scalar values such as names, operators, and numbers. *Reference Elements* point to other elements, also across multiple AASs. *Submodel Element Collections (SMCs)* represent structured elements such as actions or predicates, and *Submodel Element Lists (SMLs)* represent homogeneous, optionally ordered lists. These structures can express both atomic planning information and nested expressions such as logical preconditions or arithmetic costs.

2.3 Related Work

Several approaches represent planning knowledge above raw PDDL and generate planner-readable artifacts. itSIMPLE uses Unified Modeling Language (UML) and Object Constraint Language (OCL) for planning-domain engineering and maps models to PDDL (Vaquero et al., 2013; Vaquero et al., 2007); recent SysML work pursues similar generation work-

flows (Nabizada et al., 2024). Kootbally et al. represent PDDL structures in OWL and Extensible Markup Language (XML) Schema for robotic assembly and generate files for kitting and replanning (Kootbally et al., 2015). These works show the value of representations above textual PDDL, but are often tied to specific notations, applications, or PDDL-oriented structures.

Semantic web service composition approaches are also predecessors. Web Ontology Language for Services (OWL-S) models Inputs, Outputs, Preconditions, and Effects, which is close to planning (Martin et al., 2004). OWLS-XPlan transforms OWL-S descriptions into PDDL 2.1 (Klusch et al., 2005); PORSCE II uses external planners for semantic service composition (Hatzi et al., 2012); and Puttonen et al. apply related ideas to factory automation (Puttonen et al., 2013). These works show that semantic action descriptions can be compiled into planning models, although their focus is service composition rather than a generic planning-information model.

Ontology-based planning-domain engineering is closest on the semantic side. Knowledge EngineeringWeb Interface (KEWI) supports ontology-centered modeling and PDDL export (Wickler et al., 2014). Ontology-mediated planning connects PDDL with OWL ontologies and rewrites combined specifications into standard planning tasks (John and Koopmann, 2023). Malburg et al. transform semantic web services and domain ontologies into PDDL for manufacturing planning and scheduling (Malburg et al., 2023). However, their source model is application-specific. Plato generalizes the ontology representation, while Pallas transfers the generic planning-information model to the AAS.

On the AAS side, Bernhard et al. use AASs for AI planning with planning-related submodels and distributed skill descriptions (Bernhard et al., 2024). Their conditions include logical and arithmetic terms, but richer numerical conditions and durative actions remain limited, and PDDL generation is manual. Recent AAS-to-PDDL work embeds skill and capability descriptions in broader planning and execution architectures (Weber et al., 2025). These approaches are close in motivation, but either lack automated transformation tooling or do not provide all details needed for domain generation.

Pallas is also related to AAS-based capability and skill modeling. The Capability-Skill-Service model distinguishes abstract capabilities, invocable skills, and externally offered services (Köcher et al., 2023). The Industrial Digital Twin Association (IDTA) Capability Description Submodel standardizes capability-related information (Industrial Digital Twin Association (IDTA), 2026), and Perzylo et al. discuss capability-based semantic interoperability for manufacturing resources (Perzylo et al., 2019). Such descriptions support matching resources to process steps, but they do not by themselves provide a complete source model for representing planning domains.

2.4 Positioning of Pallas

Existing work shows that planning knowledge engineering has moved beyond hand-written PDDL. Ontology-based approaches provide semantic models and reasoning support; AAS-based approaches provide standardized industrial exchange and proximity to operational assets. Pallas targets their intersection. It is more generic than domain-specific semantic-to-PDDL approaches, more planning-complete than capability-only AAS models, and more directly integrated into AAS infrastructures than Plato. In particular, it represents not only what an asset can do, but also the planning structures required for PDDL generation: types, entities, predicates, actions, preconditions, effects, functions, costs, metrics, initial states, and goals. Its role is not to replace Plato or PDDL, but to provide an AAS-native semantic source model from which planner-specific PDDL artifacts can be generated.

3 REQUIREMENTS AND METHODOLOGY

Pallas was developed as a requirements-driven AAS model for semantic planning knowledge, using requirements derived and extended from Plato. R1–R6 cover the core constructs required for planning-domain representation, R7–R8 address planner-specific PDDL generation, and R9 captures the modularity required by asset-centric AAS environments. The first eight requirements correspond to the Plato baseline but are adapted from an ontology schema to an AAS model. As with Plato, the goal is not complete coverage of the full PDDL family, but a pragmatic, extensible core for common planning-domain modeling.

R1: Typed planning entities and type hierarchies. Pallas must represent planning objects, constants, and resources with planning-relevant names and types. Type information is needed so generated artifacts can restrict valid arguments for action, predicate, and function parameters. Type hierarchies must express generalization and specialization, for exam-

ple when several specific resource types should be usable wherever a general resource is expected.

R2: Predicates with ordered parameters. Pallas must represent predicates for describing initial states, goals, preconditions, and effects. Without explicit predicate definitions, the model could not describe when an action is applicable or how its execution changes the world. Predicate definitions need names and ordered typed parameters because PDDL interprets arguments positionally, so `on(?x, ?y)` differs from `on(?y, ?x)`.

R3: Actions with parameters, preconditions, effects, and optional temporal information. The model must represent action-like capabilities that transform the world state. Each action requires typed ordered parameters so it can be instantiated with concrete planning entities during solving. Preconditions state the facts or expressions required for applicability, while effects describe resulting changes. For temporal domains, Pallas should also support durations and time-scoped conditions or effects that apply at the start, at the end, or throughout execution.

R4: Arithmetic expressions for costs and durations. Planning domains often include numeric information such as costs, durations, resource consumption, or production quantities. Such information is also needed when plans should be efficient or suitable according to quantitative criteria, not merely feasible. Pallas must therefore represent both static values and composed arithmetic expressions involving functions, parameters, and nested expressions. This is required because relevant values may depend on resources, materials, distances, machine configurations, or other action parameters.

R5: Optimization metrics. The model must support optimization objectives, such as minimizing cost or maximizing utility. Many applications require choosing among several valid plans that differ in characteristics, such as production time, energy consumption, material use, or overall cost. A metric must define the arithmetic expression to evaluate and whether it should be minimized or maximized. Defining multiple metrics and one selecting one during transformation to PDDL should also be allowed.

R6: Functions and initial numeric values. Pallas must represent numeric functions for parameter-dependent mappings, such as transport costs between locations, processing durations for machine-product combinations, material costs, or energy consumption. These mappings cannot always be encoded as scalar properties because their values depend on the concrete arguments used in a planning task. Function signatures should define expected parameters, while initial values define known values for relevant parameter combinations. The same functions should be usable in action costs, duration expressions, numeric effects, and metrics.

R7: Numeric discretization for planner compatibility. Because many planners provide limited numeric support, Pallas should represent bounded numeric values so they can be transformed into discrete symbolic alternatives. This allows compact numeric source information to be compiled into simplified PDDL using minimum value, maximum value, and step size. It also avoids forcing modelers to manually enumerate symbolic values when they can be derived systematically.

R8: Transformation into planner-specific PDDL fragments. Pallas must support tools and information for generating PDDL artifacts adapted to target-planner capabilities. Planner support differs considerably for numeric expressions, temporal constructs, negative preconditions, quantified expressions, and complex logical conditions. The source model should therefore not be tied to one generated PDDL variant. Instead, transformation tools should be able to preserve, simplify, or rewrite information for different planners, reducing the need to maintain several manual domain versions.

R9: Modular and distributed representation across multiple AASs. Pallas must allow planning knowledge to be distributed across multiple AASs. In AAS-based environments, individual assets frequently have their own shells, so planning knowledge should not have to reside in one monolithic model. Asset-specific actions and resources should be maintainable in or near the corresponding asset shell. The distributed information must still be combinable into one coherent planning representation.

The methodology followed an artifact-oriented design process. First, planning information and its structure were identified from Plato and adapted to AAS integration needs. Second, Pallas was designed as an AAS model using standard submodel structures. Third, KTF was extended to support automatic transformation from and to Pallas models. Finally, the approach was evaluated through proof-of-concept transformations and an exploratory user study.

4 PALLAS

Pallas is an AAS-based model for semantic planning knowledge stored in one or more planning-information submodels. The model is open source and available under the MIT license on GitHub[1]; the

[1] https://github.com/user140613/Pallas

repository also provides a more detailed specification that cannot be included here in full due to space constraints.

4.1 Overview

Pallas uses four AAS element types. Properties store atomic values such as names, operators, numbers, and type indicators. Reference Elements connect elements, for example an entity to its type or an action to its conditions. SMCs represent structured planning elements such as actions, predicates, and entities. SMLs represent collections; ordered SMLs are used where parameter or argument order matters.

The model is conceptually close to Plato but implemented differently. Plato represents planning knowledge as RDF/OWL individuals and relations, whereas Pallas represents it as AAS submodel elements and references. This makes Pallas directly integrable into AAS-based environments while preserving the ability of PDDL generation.

Figure 1 visualizes the model. Pallas stores planning knowledge in top-level SMLs and a top-level `Name` property. `Types` stores planning types; `PlanningConstants` and `PlanningObjects` store reusable and problem-specific entities; `Predicates` stores predicate signatures; `Functions` and `FunctionValues` store numeric functions and initial values; `InitialState` and `GoalState` store initial and goal statements; `Metrics` stores optimization objectives; and `Actions` stores actions. Each list may contain an arbitrary number of corresponding structured elements, for example several `Action` SMCs inside `Actions`.

4.2 Types, Planning Entities, and Parameters

Each type is an SMC with a name and an optional reference to its supertype. This bottom-up representation keeps the hierarchy explicit without redundantly storing subtype relations.

Planning constants and planning objects are structurally similar SMCs with a name and type reference. Constants belong to the reusable domain, such as fixed resources or locations; objects belong to a problem instance, such as workpieces or orders. This maps well to PDDL constants and problem objects.

Pallas also supports numeric planning entities through optional `MinNumericValue`, `MaxNumericValue`, and `StepSize` properties. During transformation, KTF can derive discrete alternatives, addressing R7 while keeping the source model compact. This is useful when the source knowledge is naturally numeric but a target planner requires symbolic values.

Parameters used in predicates, statements, actions, functions, and function calls are represented uniformly as parameter SMCs with a name and a type reference.

4.3 Predicates, Statements, and States

A predicate SMC contains a name and an ordered SML of parameters. It defines the signature of a boolean statement without binding it to concrete entities or action parameters.

Statements instantiate predicates for initial-state facts, goal requirements, preconditions, or effects. A statement SMC references a predicate and provides an ordered list of parameters or concrete entities. For example, an action effect may combine action parameters with constants from the planning domain. In preconditions and effects, a statement may also contain a temporal operator specifying whether the condition or effect applies at the start, at the end, or over the whole action duration.

The top-level `InitialState` list contains statements true initially, while `GoalState` contains statements that must hold for a solution.

4.4 Actions and Logical Expressions

Each action is an SMC with a name, ordered parameters, preconditions, effects, costs, and optionally durations for durative actions.

Preconditions and effects are represented as tree-structured logical-expression SMCs. Each expression stores an operator and, where needed, ordered nested expressions. Pallas supports `AND`, `OR`, `NOT`, `IMPLY`, `EXIST`, `FORALL`, and `WHEN`. Conjunctions and disjunctions contain multiple children, negation one child, and implication two children (i.e., child1 implies child2). Quantified expressions use additional parameter information to specify the relevant type, for example in a `forall` expression over blocks. Statements form the leaves of the logical tree.

A cost SMC references the function value to modify and the relevant parameters. A function can be increased, decreased, scaled up, scaled down, or assigned a value. The value is an arithmetic-expression SMC as described in Section 4.5. Numeric effects that are not optimization costs are still modeled with the `Cost` SMC. Additionally, the cost SMC is also usded to represent durations.

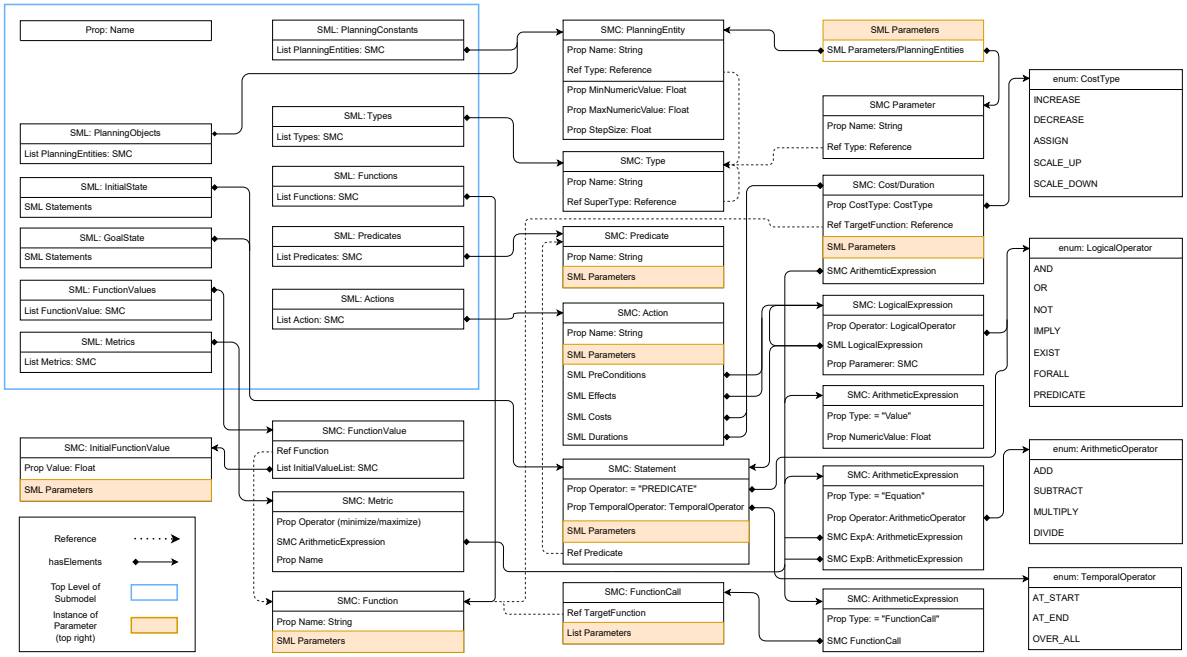

Figure 1: Overview of the Pallas model for representing semantic planning information in Asset Administration Shells. Due to space constraints, some details are omitted.

4.5 Functions, Arithmetic Expressions, and Metrics

Numeric functions are stored in `Functions`. Each function has a name and ordered parameters. Initial function values, usually part of the PDDL initial state, are stored separately in `FunctionValues`; each function value SMC references a function and specifies values for concrete parameter combinations. This separation keeps the reusable function signature distinct from problem-specific numeric facts.

Arithmetic expressions are used for action costs, durations, and metrics. They are nested SMCs of type `Value`, `FunctionCall`, or `Equation`. A value stores a number, a function call references a function with ordered parameters, and an equation combines two expressions with an arithmetic operator. Equations follow binary expression trees (Issel, 1984) and support addition, subtraction, multiplication, and division.

Optimization metrics are stored in `Metric`. A metric contains a name, an operator for minimization or maximization, and the arithmetic expression to optimize. This supports objectives such as minimizing cost, maximizing value, or combining multiple functions in a weighted expression.

4.6 Distributed Modeling Across AASs

A central Pallas feature is native distribution of planning information across several AASs. Top-level SMLs need not be stored in one shell; several shells may also contain the same top-level category, which KTF combines during PDDL transformation. For example, each machine AAS may contain an `Actions` list with the actions provided by that machine. This avoids a monolithic planning model that separates asset-specific planning knowledge from the digital representation where related asset information is already maintained.

5 TRANSFORMATION AND TOOL SUPPORT WITH KTF

Pallas is operationalized through the Knowledge Transformation Framework (KTF, 2026) . KTF uses a Consumer-Intermediary-Producer (CIP) architecture: consumers parse source representations and map them to an intermediate planning representation, transformations adapt this representation to target requirements, and producers serialize it into target formats. This design separates source-model maintenance from planner-specific generation.

For Pallas, the AAS consumer reads AAS files, identifies configured planning-information submod-

els, parses their SMLs and SMCs, resolves references, and builds the intermediate representation. The consumer supports distributed modeling by accepting several AAS files. During consumption, these elements are merged into one planning model. This is essential for satisfying R9.

The PDDL producer then generates domain and problem files from the intermediate representation. The generated domain contains the planning types, constants, predicates, functions, and actions derived from Pallas. The generated problem contains problem-specific objects, initial state, goal state, and metrics where applicable. The problem definition can either come from the Pallas model itself or be configured during transformation.

Since KTF uses an intermediate planning representation, Pallas can also interoperate with Plato within their shared conceptual constructs. A Plato ontology can be consumed and produced as a Pallas AAS model. This does not imply that all semantics are preserved in every representation, but it provides a practical migration and comparison path for planning-relevant knowledge.

KTF also supports planner-specific transformations. Planners differ in their support for PDDL fragments, such as numeric expressions, negative preconditions, temporal constructs, and complex logical conditions. KTF can replace unsupported numeric expressions with precomputed values or rewrite constructs for a selected planner. In the context of Pallas, this means that the AAS model can remain the stable source representation while generated PDDL artifacts are adapted to planner capabilities.

Besides generation, KTF provides validation and feedback during parsing. The AAS consumer can detect missing required elements, unresolved references, incomplete parameter lists, invalid expression structures, and constructs that cannot be interpreted unambiguously. Such feedback is important because deeply nested AAS structures are difficult to inspect manually. KTF can also report statistics, such as the number of actions, predicates, preconditions, effects, and other model elements, which helps compare model versions or identify unusually complex model parts.

KTF is therefore not merely a file converter. It provides the technical link between Pallas as an AAS-native source representation and planner-specific artifacts. This separation allows Pallas to focus on maintainable knowledge representation and allows KTF to handle validation, composition, transformation, and generation.

6 EVALUATION

The evaluation has two parts. First, a proof-of-concept evaluation checks whether Pallas can represent selected planning constructs and whether KTF can generate usable PDDL. Second, an exploratory user study compares Pallas and Plato for modeling and modification tasks. The goal is initial evidence for feasibility and usability, not complete validation for all planning domains or industrial settings.

6.1 Proof-of-Concept Transformation Evaluation

The transformation evaluation mirrors the proof-of-concept strategy used for Plato. Four Pallas models were built to excerise different model parts of Pallas: **BlocksWorld**, **TemporalBlocksWorld**, **SharedProduction**, and **3DPrinting**.

BlocksWorld: This compact classical domain covers typed entities, predicates, ordered parameters, actions, preconditions, and effects. It serves as an easily inspectable baseline.

TemporalBlocksWorld: This model extends Blocks World with temporal information and checks representation and generation of temporal constructs.

SharedProduction: This model represents a joint production setting involving the SmartFactoryOWL[2] and the SmartFactoryKL[3]. It was developed in the [redacted for blind review] project, where PDDL-based planning coordinates production across factory boundaries. The model includes production and transport resources, warehouses, 3D printers, cameras, materials, workpieces, quality checks, and cost-related functions. It is split across two AASs to evaluate R9. Two generated PDDL variants are examined: one retains numeric action costs, while the other replaces them with precomputed costs for a planner without numeric support. This tests Pallas-based generation for planners with different capabilities.

3DPrinting: This simplified 3D-printing process includes printers, materials, print times, temperatures, material costs, energy costs, and a composed metric. It evaluates numeric constants and discretization by converting modeled numeric ranges, such as nozzle temperatures and print durations, into discrete PDDL constants and associated function values.

Together, the examples cover the requirements from Section 3. R1, R2, and the classical part of R3 are covered by all domains; the temporal part of R3 by `TemporalBlocksWorld`; R4–R6 by

[2]https://www.smartfactory-owl.de/en/

[3]https://smartfactory.de/

SharedProduction and 3DPrinting; R7 and R8 by 3DPrinting and KTF's planner-specific generation; and R9 by SharedProduction and distributed consumption.

For all four models, KTF produced valid PDDL domain and problem files without manual post-processing and the domains contained the expected constructs. The generated artifacts were assessed in two stages. First, all PDDL domain and problem files were validated with VAL version 4[4], which reported no issues; the authors also manually inspected the generated files. Second, each task was solved with a planner suitable for their used PDDL fragments. BlocksWorld and SharedProduction with substituted numeric expressions used Fast Downward v22.12[5]. TemporalBlocksWorld used Temporal Fast Downward v0.4[6]. SharedProduction with numeric expressions and 3DPrinting used ENHSP-20 v0.21.0[7]. In all cases, the planner produced a plan that was successfully validated by VAL version 4.

These findings show that generated PDDL artifacts are syntactically valid and usable with existing planning tools. They also show that Pallas can act as a source model for classical, temporal, numeric, and planner-simplified PDDL variants.

6.2 User Study Comparing Pallas and Plato

The second evaluation part is an exploratory user study comparing AAS-based modeling with Pallas and ontology-based modeling with Plato. Participants modified the same Blocks World planning concepts in AASX Package Explorer[8] for Pallas/AAS and Prot'eg'e (Musen and Protégé Team, 2015) for Plato. The focus lied on knowledge engineering, not planner performance.

Nine people participated. After an introductory example in both tools, they had one hour for three tasks: add a predicate indicating whether a location is occupied; modify the preconditions and effects of put-down-on-ground and pick-up-from-ground to use it; and add a location parameter to the existing onGround predicate. Participants then answered a questionnaire on experience, completion, preferences, suitability ratings, and qualitative feedback.

The background data were biased toward ontologies. All participants had heard of ontologies, five had worked with them, and three had done so within the previous year. In contrast, five had never heard of AASs, two had only heard of them, and two had worked with them. Tool experience was similar: four had used Prot'eg'e, three had heard of it, and only the two AAS-experienced participants had used AASX Package Explorer. This background must be considered when interpreting tool preferences.

Despite this bias, all participants completed the first two tasks in both tools. The third task was completed by three participants in Prot'eg'e and two in AASX Package Explorer, mainly due to task difficulty and the time limit. Perceived suitability differed more strongly: eight of nine participants selected Prot'eg'e as more suitable. On a scale from 1 to 5, Prot'eg'e averaged 4.0 and AASX Package Explorer 2.33. Model preference between Plato and Pallas remained inconclusive because most participants could not state a preference.

Qualitative feedback reinforced this result. Participants described Prot'eg'e as more intuitive, structured, and navigable, especially with prior ontology experience; its hierarchy helped them locate and modify elements. They nevertheless noted limitations such as missing auto-completion or unintuitive interface details. AASX Package Explorer was often described as cluttered, nested, and harder to navigate; participants reported more effort in locating objects, classes, and attributes and in verifying changes. A minority appreciated the hierarchical representation and relation inspection. Overall, comments suggest that tool differences overshadowed model differences, and that participants could not formulate proper opinions about Plato versus Pallas independently of the tools.

The study therefore gives a balanced result. Pallas-based planning knowledge could be modified with current AAS tooling, but authoring usability was weaker than ontology-based modeling with Prot'eg'e in this setting. Because the sample was small, participants were more familiar with ontologies, and the AAS tool was not designed for complex planning expressions, the study indicates feasibility and tooling challenges rather than a definitive comparison between AAS and ontology modeling.

7 DISCUSSION

The evaluation suggests that Pallas can represent selected core planning constructs and that KTF can generate PDDL from them. The main contribution, however, is the separation of planning knowledge maintenance from planner-oriented syntax in an AAS-native form.

[4]https://github.com/KCL-Planning/VAL
[5]https://github.com/aibasel/downward
[6]https://github.com/roveri-marco/tfd
[7]https://github.com/hstairs/enhsp
[8]https://github.com/eclipse-aaspe/package-explorer

7.1 Benefits Compared to Direct PDDL Modeling

Direct PDDL modeling requires domain experts to encode knowledge in a language designed for planners, which complicates maintenance as domains evolve. Pallas shifts the source representation to AAS submodels, where planning concepts can be referenced, reused, and transformed.

This separation improves maintainability. Recurring concepts can be modeled once and referenced in multiple places; changes to actions, predicates, functions, or metrics can be made in the AAS source and propagated through transformation; and planner-specific restrictions can be handled during generation rather than manually encoded in the source model. This is particularly useful when the same domain must be produced for planners with different support for numeric or temporal constructs. Thus, PDDL becomes a generated artifact instead of the primary maintenance format.

7.2 Benefits Compared to Plato

Pallas and Plato are complementary. Plato benefits from RDF/OWL expressiveness, mature ontology tools, ontology validation, and reasoning, which are valuable when organizations already use knowledge graphs or need semantics beyond the AAS metamodel.

Pallas optimizes for AAS-based industrial environments and asset-centric maintenance. If a machine, workstation, or transport resource already has an AAS, planning information for its services can be stored in or near that shell instead of in a separate planning ontology. This supports distributed ownership, while KTF combines the submodels for planning.

The user study highlights the trade-off. Prot'eg'e was perceived as more convenient for editing the evaluated model, whereas AASX Package Explorer was harder for nested planning expressions. This does not invalidate Pallas, but shows the need for better authoring support, including dedicated editors, expression visualization, and validation feedback directly integrated into the development environment.

7.3 Distinctive Features of Pallas

The first distinctive feature is AAS-native representation. Pallas uses standard AAS structures instead of an external planning-specific syntax, allowing exchange, storage, and processing in AAS-based infrastructures. The second feature is modular and distributed modeling. Pallas can represent a planning domain across several AASs and relies on KTF to merge the information during consumption, matching industrial settings with separate shells for assets, machines, products, and services.The third feature is compilation-oriented semantics. Pallas stores planning information specifically that can compiled for PDDL generation in planner and problem specific artifacts. The fourth feature is interoperability with the Plato/KTF ecosystem. Because Pallas and Plato can be mapped into the same intermediate representation, transformation logic can be reused where their conceptual cores overlap.

7.4 Limitations and Open Issues

Several limitations remain. First, the proof-of-concept examples are curated to cover requirements and demonstrate features. They show feasibility, but not sufficiency for all relevant planning domains; larger independently developed Pallas models are needed.

Second, transformation success does not prove scalable authoring usability. Deeply nested SMCs and SMLs can be hard to inspect and edit with general-purpose AAS tools. The user study suggests that current tooling may hinder complex expression modeling. This limitation concerns authoring support as much as the model itself, because dedicated editors could hide low-level nesting and expose task-oriented forms.

Third, the user study is exploratory. It involved nine participants with more prior exposure to ontologies than AASs and compared concrete tools rather than only abstract modeling formalisms. The preference for Prot'eg'e therefore cannot be attributed solely to Plato or ontologies in general. Future and more extensive studies should separate model usability from tool usability more carefully and use participants with balanced prior experience.

Fourth, Pallas covers a pragmatic subset of planning knowledge rather than the full PDDL family. Additional constructs may be required for richer temporal planning, constraints, preferences, hierarchical planning, uncertainty, or planner-specific extensions.

Finally, distributed modeling needs evaluation in realistic AAS landscapes. Splitting information across shells raises questions about versioning, reference resolution, responsibility boundaries, cross-shell validation, and consistency when assets change independently.

8 CONCLUSION AND FUTURE WORK

This paper presented Pallas, a generic AAS-based model for semantic planning knowledge and PDDL generation. Pallas addresses the need for a maintainable source representation by modeling planning concepts such as types, constants, predicates, functions, and actions with AAS constructs such as submodels, SMLs, SMCs, Properties, and Reference Elements.

Pallas was guided by requirements derived from PDDL modeling constructs, prior semantic-to-PDDL transformation work, planner compatibility, and modular distributed representation across multiple AASs. It is supported by KTF, which consumes Pallas models, merges distributed planning information, applies configured planner-specific transformations, and generates PDDL domain and problem files.

The evaluation provides initial feasibility evidence. Four proof-of-concept models cover the requirements and exercise classical, temporal, numeric, and discretized planning constructs. The exploratory user study shows that Pallas-based planning knowledge can be modified with current AAS tooling, but that participants preferred Plato with Prot'eg'e in the evaluated setting, likely due to tool maturity and prior experience.

Future work should extend evaluation to larger industrial case studies and more complex domains. Dedicated Pallas authoring tools could visualize nested logical and arithmetic expressions, guide modeling, and provide direct validation feedback. Future work should further investigate distributed planning knowledge across multiple AASs, including reference management, consistency checking, and synchronization with asset states. Pallas could also be integrated more deeply with existing AAS templates for capabilities, skills, and services, and compared more thoroughly with Plato to determine which representation best fits which modeling context.

ABBREVIATIONS

The following abbreviations are used in this manuscript:

AAS	Asset Administration Shell
AI	Artificial Intelligence
CIP	Consumer-Intermediary-Producer
I4.0	Industry 4.0
IDTA	Industrial Digital Twin Association
IoT	Internet of Things
KEWI	Knowledge EngineeringWeb Interface
KTF	Knowledge Transformation Framework
LLM	Large Language Model
OCL	Object Constraint Language
OWL-S	Web Ontology Language for Services
OWL	Web Ontology Language
PDDL	Planning Domain Definition Language
RDF	Resource Description Framework
SMC	Submodel Element Collection
SML	Submodel Element List
UML	Unified Modeling Language
XML	Extensible Markup Language

ACKNOWLEDGEMENTS

[redacted for blind review]

USE OF GENERATIVE AI

ChatGPT (OpenAI, 2026) was used to polish the wording and correct spelling and grammar mistakes for the sections in this paper. Each section was first written by the authors without the use of any AI tools before being passed to the Large Language Model (LLM) for polishing. All LLM-adjusted texts were carefully checked sentence by sentence for accuracy by the authors. No graphs, figures, code, data, or citations were AI-generated.

REFERENCES

Alnazer, E. and Georgievski, I. (2023). Understanding real-world AI planning domains: A conceptual framework. In Aiello, M., Barzen, J., Dustdar, S., and Leymann, F., editors, *Service-Oriented Computing - 17th Symposium and Summer School, SummerSOC 2023, Heraklion, Crete, Greece, June 25 - July 1, 2023, Revised Selected Papers*, volume 1847 of *Communications in Computer and Information Science*, pages 3–23. Springer.

Alterovitz, R., Koenig, S., and Likhachev, M. (2016). Robot planning in the real world: Research challenges and opportunities. *AI Mag.*, 37(2):76–84.

Ammar, S. and Bhiri, M. T. (2018). Automatic planning: From event-b to PDDL. In Abdelwahed, E. H., Bellatreche, L., Benslimane, D., Golfarelli, M., Jean, S., Méry, D., Nakamatsu, K., and Ordonez, C., editors, *New Trends in Model and Data Engineering - MEDI 2018 International Workshops, DETECT, MEDI4SG, IWCFS, REMEDY, Marrakesh, Morocco, October 24-26, 2018, Proceedings*, volume 929 of *Communica-*

tions in Computer and Information Science, pages 247–254, Cham Switzerland. Springer.

Bartheye, O. and Jacopin, E. (2009). A real-time pddl-based planning component for video games. In Darken, C. and Youngblood, G. M., editors, *Proceedings of the Fifth Artificial Intelligence and Interactive Digital Entertainment Conference, AIIDE 2009, October 14-16, 2009, Stanford, California, USA*. The AAAI Press.

Bernhard, A. T., Blumhofer, B., Ruskowski, M., Wagner, A., Luxenburger, A., and Porta, D. (2024). Leverage asset administration shells to support artificial intelligence planning. In *29th IEEE International Conference on Emerging Technologies and Factory Automation, ETFA 2024, Padova, Italy, September 10-13, 2024*, pages 1–8. IEEE.

Bresina, J. L., Jónsson, A. K., Morris, P. H., and Rajan, K. (2005). Activity planning for the mars exploration rovers. In Biundo, S., Myers, K. L., and Rajan, K., editors, *Proceedings of the Fifteenth International Conference on Automated Planning and Scheduling (ICAPS 2005), June 5-10 2005, Monterey, California, USA*, pages 40–49. AAAI.

Feigenbaum, E. A. (1977). The art of artificial intelligence: Themes and case studies of knowledge engineering. In Reddy, R., editor, *Proceedings of the 5th International Joint Conference on Artificial Intelligence. Cambridge, MA, USA, August 22-25, 1977*, pages 1014–1029. William Kaufmann.

Fox, M. and Long, D. (2003). Pddl2.1: An extension to pddl for expressing temporal planning domains. *J. Artif. Intell. Res. (JAIR)*, 20:61–124.

Gilchrist, A. (2016). *Industry 4.0*. APress, Berlin, Germany, 1 edition.

Green, C. (1969). Application of Theorem Proving to Problem Solving. In *1st IJCAI Proc.*, IJCAI'69, page 219–239, San Francisco, CA, USA. Morgan Kaufmann Publishers Inc.

Haslum, P. (2006). *Admissible Heuristics for Automated Planning*. PhD thesis, Linköping University, KPLAB - Knowledge Processing Lab, The Institute of Technology.

Haslum, P., Lipovetzky, N., Magazzeni, D., and Muise, C. (2019). *An Introduction to the Planning Domain Definition Language*. Synthesis Lectures on Artificial Intelligence and Machine Learning. Morgan & Claypool Publishers, California, USA.

Hatzi, O., Vrakas, D., Nikolaidou, M., Bassiliades, N., Anagnostopoulos, D., and Vlahavas, I. (2012). An integrated approach to automated semantic web service composition through planning. *IEEE Transactions on Services Computing*, 5(3):319–332.

Helmert, M. (2009). Concise finite-domain representations for PDDL planning tasks. *Artif. Intell.*, 173(5-6):503–535.

Industrial Digital Twin Association and Plattform Industrie 4.0 (2024). *Asset Administration Shell: Reading Guide*. Industrial Digital Twin Association (IDTA). Accessed 16.05.2026.

Industrial Digital Twin Association (IDTA) (2026). *Capability Description*. Accessed 18.05.2026.

Issel, W. (1984). Aho, a.v., j. e. hopcroft, j. d. ullman: Data structures and algorithms. addison-wesley amsterdam 1983. 436 s. *Biometrical Journal*, 26(4):390–390.

John, T. and Koopmann, P. (2023). Planning with ontology-enhanced states using problem-dependent rewritings. In Umbrico, A. and Sanfilippo, E. M., editors, *Proceedings of the 1st International Planning and Ontology Workshop co-located with The 33rd International Conference on Automated Planning and Scheduling (ICAPS 2023), Prague, Czech Republic, July 10, 2023*, CEUR Workshop Proceedings. CEUR-WS.org.

Klusch, M., Gerber, A., and Schmidt, M. (2005). Semantic web service composition planning with owls-xplan. *AAAI Fall Symposium - Technical Report*.

Köcher, A., Belyaev, A., Hermann, J., Bock, J., Meixner, K., Volkmann, M., Winter, M., Zimmermann, P., Grimm, S., and Diedrich, C. (2023). A reference model for common understanding of capabilities and skills in manufacturing. *Autom.*, 71(2):94–104.

Kootbally, Z., Schlenoff, C., Lawler, C., Kramer, T., and Gupta, S. (2015). Towards robust assembly with knowledge representation for the planning domain definition language (pddl). *Robotics and Computer-Integrated Manufacturing*, 33:42–55. Special Issue on Knowledge Driven Robotics and Manufacturing.

KTF (2026). KTF: A framework for semantic knowledge transformation. To be submitted for review at KEOD 2026. Authors removed for double-blind review since we are referencing our own work. Anonymized preprint available at https://github.com/user140613/Preprints.

Malburg, L., Klein, P., and Bergmann, R. (2023). Converting semantic web services into formal planning domain descriptions to enable manufacturing process planning and scheduling in industry 4.0. *Eng. Appl. Artif. Intell.*, 126:106727.

Marrella, A. (2019). Automated planning for business process management. *J. Data Semant.*, 8(2):79–98.

Martin, D. L., Paolucci, M., McIlraith, S. A., Burstein, M. H., McDermott, D. V., McGuinness, D. L., Parsia, B., Payne, T. R., Sabou, M., Solanki, M., Srinivasan, N., and Sycara, K. P. (2004). Bringing semantics to web services: The OWL-S approach. In Cardoso, J. and Sheth, A. P., editors, *Semantic Web Services and Web Process Composition, First International Workshop, SWSWPC 2004, San Diego, CA, USA, July 6, 2004, Revised Selected Papers*, Lecture Notes in Computer Science, pages 26–42. Springer.

McCluskey, T. L., Vallati, M., and Franco, S. (2017). Automated planning for urban traffic management. In Sierra, C., editor, *Proceedings of the Twenty-Sixth International Joint Conference on Artificial Intelligence, IJCAI 2017, Melbourne, Australia, August 19-25, 2017*, pages 5238–5240. ijcai.org.

Muscettola, N., Nayak, P. P., Pell, B., and Williams, B. C. (1998). Remote agent: To boldly go where no AI system has gone before. *Artif. Intell.*, 103(1-2):5–47.

Musen, M. A. and Protégé Team (2015). The protégé project: A look back and a look forward. *AI Matters*, 1(4):4–12.

Nabizada, H., Jeleniewski, T., Gehlhoff, F., and Fay, A. (2024). Model-based workflow for the automated generation of pddl descriptions. In *2024 IEEE 29th International Conference on Emerging Technologies and Factory Automation (ETFA)*, pages 01–04.

OpenAI (2026). Chatgpt (version 5.5). Large language model.

Perzylo, A., Grothoff, J., Lucio, L., Weser, M., Malakuti, S., Venet, P., Aravantinos, V., and Deppe, T. (2019). Capability-based semantic interoperability of manufacturing resources: A BaSys 4.0 perspective. *IFAC-PapersOnLine*, 52(13):1590–1596.

Plato (2026). Plato: Generic ontology-based representation of semantic planning knowledge for automatic PDDL generation. To be submitted for review at KEOD 2026. Authors removed for double-blind review since we are referencing our own work. Anonymized preprint available at https://github.com/user140613/Preprints.

Puttonen, J., Lobov, A., and Martinez Lastra, J. L. (2013). Semantics-based composition of factory automation processes encapsulated by web services. *Industrial Informatics, IEEE Transactions on*, 9:2349–2359.

Russell, S. and Norvig, P. (2020). *Artificial Intelligence: A Modern Approach (4th Edition)*. Pearson, London, United Kingdom.

Studer, R., Benjamins, V. R., and Fensel, D. (1998). Knowledge engineering: Principles and methods. *Data Knowl. Eng.*, 25(1-2):161–197.

Vaquero, T. S., Romero, V., Tonidandel, F., and Silva, J. R. (2007). itsimple 2.0: An integrated tool for designing planning domains. In *International Conference on Automated Planning and Scheduling*.

Vaquero, T. S., Silva, J. R., Tonidandel, F., and Beck, J. C. (2013). itsimple: towards an integrated design system for real planning applications. *The Knowledge Engineering Review*, 28:215 – 230.

Wally, B., Vyskočil, J., Novák, P., Huemer, C., Šindelář, R., Kadera, P., Mazak, A., and Wimmer, M. (2019). Production planning with iec 62264 and pddl. In *2019 IEEE 17th International Conference on Industrial Informatics (INDIN)*, volume 1, pages 492–499, New York, NY, USA. IEEE.

Weber, H., Glück, R., and Krebs, F. (2025). Leveraging semantic interoperability in industry 4.0: An opportunity for automated task planning in discrete manufacturing processes. In *Deutscher Luft- und Raumfahrtkongress (DLRK)*.

Wickler, G., Chrpa, L., and McCluskey, T. L. (2014). Ontological support for modelling planning knowledge. In Fred, A. L. N., Dietz, J. L. G., Aveiro, D., Liu, K., and Filipe, J., editors, *Knowledge Discovery, Knowledge Engineering and Knowledge Management - 6th International Joint Conference, IC3K 2014, Rome, Italy, October 21-24, 2014, Revised Selected Papers*, Communications in Computer and Information Science, pages 293–312. Springer.